

PYTHON STRINGS

1. String Introduction:

A string is a sequence of characters, and it is one of the most commonly used data types in programming. In most programming languages, strings are used to represent text. In Python, strings are immutable, meaning that once a string is created, it cannot be changed. In Python, a string is a sequence of characters enclosed in either single quotes (`' '`), double quotes (`" "`), or triple quotes (`''' '''` or `""" """`). It serves as one of the fundamental data types in Python.

Example:

```
# Creating strings
single_quoted_string = 'Hello, World!'
double_quoted_string = "Python Strings"
triple_quoted_string = '''Triple quotes allow
strings to span multiple lines.'''
```

2. String Object Basics:

In Python, strings are objects of the `str` class. You can perform various operations on strings, such as accessing individual characters, slicing, and more.

1. Indexing:

In Python, indexing refers to the process of accessing individual elements in a data structure, such as a list, tuple, or string, using their position or index within the data structure. The index is a numerical value that represents the position of an element in the sequence, and it usually starts from 0.

```
my_string = "Python"

# Accessing individual characters
first_char = my_string[0]    # 'P'
third_char = my_string[2]    # 't'
last_char = my_string[-1]    # 'n'
```

```
print(first_char, third_char, last_char)
```

2. Slicing:

In Python, slicing is a way to extract a portion or a subsequence of elements from a sequence (like a list, tuple, or string). The syntax for slicing is generally `sequence[start:stop:step]`, where:

- `start`: The index of the first element you want in the slice.
- `stop`: The index of the first element you do not want in the slice (it's an exclusive boundary).
- `step` (optional): The step or interval between elements in the slice. If omitted, it defaults to 1.

```
my_string = "Python is powerful"

# Slicing to get substrings
substring1 = my_string[7:9]      # 'is'
substring2 = my_string[10:]     # 'powerful'
substring3 = my_string[:6]      # 'Python'

print(substring1, substring2, substring3)
```

3. Using Negative Indexing:

Negative indexing in Python allows you to access elements from the end of a sequence by using negative integers as indices. In other words, with negative indexing, you can count elements from the end of the sequence, where -1 represents the last element, -2 represents the second-to-last element, and so on.

```
my_string = "Python"

# Negative indexing to access characters from the end
second_last_char = my_string[-2]  # 'o'
```

```
last_three_chars = my_string[-3:] # 'hon'

print(second_last_char, last_three_chars)
```

4. Skipping Characters (Step):

```
my_string = "Python is fun and powerful"

# Slicing with a step to skip characters
every_second_char = my_string[::2] # 'Pto sfnadpwrfl'
reverse_string = my_string[::-1]    # 'lufrewop dna nuf s
i nohtyP'

print(every_second_char, reverse_string)
```

3. String Methods:

Python provides a variety of built-in methods for manipulating strings.

Common String Methods:

```
1. upper() and lower()
upper(): Returns a copy of the original string with all alphabetic characters converted to uppercase.
lower(): Returns a copy of the original string with all alphabetic characters converted to lowercase.

message = "Hello, World!"
uppercase_message = message.upper()
lowercase_message = message.lower()

print(uppercase_message) # Output: HELLO, WORLD!
print(lowercase_message) # Output: hello, world!

2.count()
count(substring): Returns the number of occurrences of the specified substring in the given string.
sentence = "This is a sample sentence."
count_i = sentence.count('i')
```

```
print(count_i) # Output: 2
```

3. strip()

strip(): Returns a copy of the original string with leading and trailing whitespaces removed.

```
whitespace_string = "    This is a string with leading and trailing whitespace.    "
```

```
stripped_string = whitespace_string.strip()
```

```
print(stripped_string)
```

```
# Output: "This is a string with leading and trailing white space."
```

length of a string

len(): Returns the number of characters (length) in the given string.

```
my_string = "Python is fun and powerful"
```

```
length_of_string = len(my_string)
```

```
print(length_of_string)
```

```
# Output: 28
```

4. split()

split(separator): Splits the string into a list of substrings based on the specified separator.

```
data = "Pythonlife,Kiran,123456"
```

```
data1 = data.split(',')
```

```
print(data1)
```

```
# Output: ['Pythonlife', 'Kiran', '123456']
```

5. replace()

replace(old, new): Returns a copy of the original string with occurrences of the specified "old" substring replaced with the "new" substring.

```
original_string = "Python is fun!"
```

```
modified_string = original_string.replace('fun', 'awesome')
```

```
print(modified_string) # Output: "Python is awesome!"
```

6. startswith() and endswith()

startswith(prefix): Returns True if the string starts with the specified prefix.

endswith(suffix): Returns True if the string ends with the specified suffix.

```
filename = "example.txt"
```

```
starts_with = filename.startswith("ex")
```

```
ends_with = filename.endswith(".txt")
```

```
print(starts_with) # Output: True
```

```
print(ends_with) # Output: True
```

7. find() and index()

find(substring): Returns the lowest index of the first occurrence of the specified substring. Returns -1 if the substring is not found.

index(substring): Returns the lowest index of the first occurrence of the specified substring. Raises a ValueError if the substring is not found.

```
sentence = "This is a sentence."
```

```
position_a = sentence.find('a')
```

```
position_i = sentence.index('i')
```

```
print(position_a) # Output: 8
```

```
print(position_i) # Output: 2
```

8. capitalize()

capitalize(): Returns a copy of the original string with its first character capitalized.

```
text = "python programming"
```

```
capitalized_text = text.capitalize()
```

```
print(capitalized_text) # Output: "Python programming"
```

9. `isdigit()` and `isalpha()`

`isdigit()`: Returns True if all characters in the string are digits.

`isalpha()`: Returns True if all characters in the string are alphabetic.

```
numeric_string = "12345"
```

```
alpha_string = "Python"
```

```
is_numeric = numeric_string.isdigit()
```

```
is_alpha = alpha_string.isalpha()
```

```
print(is_numeric) # Output: True
```

```
print(is_alpha)   # Output: True
```

10. `join()`

`join(iterable)`: Concatenates the elements of an iterable (e.g., a list) into a single string, using the specified separator

```
word_list = ['Hello', 'World']
```

```
joined_string = ' '.join(word_list)
```

```
print(joined_string) # Output: "Hello World"
```

4. Splitting and Joining Strings:

Splitting and joining are useful operations when dealing with strings.

Splitting:

```
# Split a string into a list of substrings
```

```
words = my_string.split(",") # Results in ['Hello', ' World!']
```

Joining:

```
# Join a list of strings into a single string
```

```
new_string = "-".join(words) # Results in 'Hello- World!'
```

5. String Format Functions:

String formatting allows you to create dynamic strings by inserting values into placeholders.

F-Strings (Python 3.6 and above):

```
formatted_str = f"My name is {name} and I am {age} years old."
```